WEB PROGRAMMING 2 • UAS

CargoTrace

Global Shipment Visibility & Tracking Platform

Final Project Brief • Requirements • Features • API • Tech Stack • Assessment

Build a software portfolio project—not just a CRUD website.

Tujuan UAS

UAS menilai kemampuan membangun, menjelaskan, menguji, dan mendemonstrasikan software.

Learning outcomes

- Menerjemahkan kebutuhan bisnis menjadi fitur dan data model.
- Membangun aplikasi Laravel dengan MySQL dan relasi yang benar.
- Mengimplementasikan business rules, REST API, dan integrasi layanan eksternal.
- Menguji, mendokumentasikan, dan melakukan deployment.

Portfolio outcomes

- Public GitHub repository dengan commit history yang bermakna.
- Aplikasi online yang dapat dicoba recruiter.
- README dan dokumentasi teknis berbahasa Inggris.
- Demo dan code ownership interview.

Prinsip penilaian

- Fungsi harus berjalan, bukan hanya tampilan.
- Mahasiswa harus memahami kode yang diserahkan.
- Kualitas engineering dinilai bersama kualitas fitur.
- Source code pihak lain wajib diatribusikan.

Problem & Solution

Domain: shipment visibility untuk perusahaan logistik dan customer.

Masalah yang diselesaikan

- Informasi shipment tersebar dan sulit dipantau.
- Customer tidak mengetahui status dan riwayat perjalanan terbaru.
- Keterlambatan dan exception tidak terlihat secara terpusat.
- Operator membutuhkan dashboard untuk memprioritaskan tindak lanjut.

Solusi CargoTrace

- Kelola shipment, carrier, lokasi, dan tracking events.
- Tampilkan timeline perjalanan dan titik lokasi pada peta.
- Deteksi keterlambatan berdasarkan aturan yang terdefinisi.
- Kelola exception dan tampilkan metrik operasional.

Scope dan batasan sistem

MVP wajib tetap kompleks secara engineering; tidak perlu meniru sistem enterprise berskala penuh.

Termasuk dalam scope

- Shipment lifecycle dan status transition.
- Tracking timeline dan peta lokasi.
- ETA/delay rule dan exception management.
- Role-based access control.
- REST API, external API, dashboard, testing, deployment.

Di luar scope wajib

- Integrasi langsung dengan pelabuhan, customs, atau carrier sungguhan.
- Pembayaran finansial atau transaksi komersial nyata.
- Pelacakan GPS real-time dari perangkat fisik.
- Machine learning/prediksi AI wajib.
- Arsitektur microservices kompleks.

Data demonstrasi boleh sintetis, tetapi harus konsisten, terdokumentasi, dan realistis.

User roles & permissions

Semua route, halaman, dan endpoint harus memeriksa otorisasi di server.

Administrator

- Kelola user dan konfigurasi.
- Kelola carrier dan lokasi.
- Lihat semua shipment dan dashboard.
- Lihat audit log.

Logistics Operator

- Buat dan perbarui shipment.
- Tambah tracking event.
- Kelola exception.
- Lihat shipment operasional.

Customer

- Lihat shipment miliknya.
- Lihat status, ETA, dan timeline.
- Lihat route pada peta.
- Lihat notifikasi dan exception yang relevan.

Fitur wajib: shipment lifecycle

Status harus dikendalikan oleh business rules, bukan input bebas tanpa validasi.

Contoh alur status yang valid



Aturan wajib

- Transisi status divalidasi di backend.
- Status terminal seperti DELIVERED tidak boleh kembali ke status awal tanpa prosedur khusus.

Data shipment minimal

- Shipment number, customer, carrier, origin, destination, cargo type, weight/volume, expected departure/arrival, current status.

Fitur wajib: tracking & timeline

Tracking event bersifat historis; jangan menimpa event lama ketika status baru ditambahkan.

Tracking event

- Shipment ID dan status/event type.
- Timestamp kejadian dan lokasi.
- Latitude/longitude jika tersedia.
- Deskripsi dan user pembuat event.
- Validasi urutan waktu dan relasi shipment.

Shipment timeline

- Urutkan event secara kronologis.
- Tampilkan status, waktu, lokasi, dan deskripsi.
- Bedakan event normal dan exception.
- Tampilkan current location dari event terbaru yang valid.
- Customer hanya dapat mengakses shipment yang menjadi haknya.

Fitur wajib: ETA, delay & exceptions

Gunakan aturan deterministik yang dapat diuji; AI/ML tidak wajib.

ETA & delay

- Bandingkan estimated arrival dengan expected arrival.
- Hitung durasi keterlambatan.
- Tetapkan threshold delay per kelompok.
- Tampilkan alasan dan nilai yang digunakan.

Exception management

- Jenis: CUSTOMS_HOLD, WEATHER_DELAY, PORT_CONGESTION, DAMAGED, MISROUTED, DOCUMENTATION_ISSUE.
- Severity dan status OPEN / INVESTIGATING / RESOLVED.
- Simpan waktu deteksi dan penyelesaian.

Contoh rule

- Jika ETA > expected arrival, tandai delayed.
- 0–6 jam: LOW; >6–24 jam: MEDIUM; >24 jam: HIGH.
- Threshold harus configurable atau mengikuti parameter kelompok.

Fitur wajib: map & external API

Integrasi harus mempunyai kegunaan bisnis dan menangani kegagalan layanan.

Map

- Leaflet.js + OpenStreetMap.
- Marker origin, destination, dan tracking points.
- Popup berisi status, waktu, dan lokasi.
- Simpan koordinat secara konsisten.
- Cantumkan attribution dan patuhi kebijakan tile provider.

External API

- Minimal satu integrasi geocoding atau weather; dua disarankan.
- Laravel HTTP Client untuk request.
- Tangani timeout, error response, dan data kosong.
- Simpan API key di environment variable, bukan repository.
- Jangan mengklaim weather sebagai bukti prediksi delay.

Dashboard, search & notification

Dashboard harus berasal dari data database, bukan angka statis.

Dashboard

- Total shipment.
- In transit, delivered, delayed.
- Open exceptions.
- Shipment by status/carrier.
- Delay/exception rate.

Search & filter

- Shipment/reference number.
- Status, carrier, origin, destination.
- Date range dan delay status.
- Sorting dan server-side pagination.

Notification & audit

- In-app notification minimal.
- Email notification sebagai advanced feature.
- Audit log untuk perubahan penting.
- Catat aktor, aksi, waktu, dan entity terkait.

Data model minimum

Mahasiswa wajib membuat ERD sendiri dan menjelaskan cardinality serta keputusan desain.

users	akun & role
shipments	shipment utama
shipment_items	detail barang
carriers	operator logistik
locations	origin/destination
tracking_events	riwayat perjalanan
shipment_exceptions	gangguan/exception
notifications	pemberitahuan
audit_logs	jejak perubahan

Tambahkan foreign key, unique constraints, indexes yang relevan, timestamps, dan soft delete hanya bila dibutuhkan.

REST API: endpoint wajib

Gunakan JSON, status code yang sesuai, validation, authentication, authorization, pagination, dan error format konsisten.

Authentication

- POST /api/register
- POST /api/login
- POST /api/logout
- GET /api/user
- Gunakan Laravel Sanctum untuk token API bila diperlukan.

Shipments

- GET /api/shipments
- POST /api/shipments
- GET /api/shipments/{id}
- PUT/PATCH /api/shipments/{id}
- DELETE /api/shipments/{id}
- GET /api/shipments/{id}/tracking
- POST /api/shipments/{id}/tracking

Related resources

- GET/POST /api/carriers
- GET/POST /api/locations
- GET /api/shipments/{id}/exceptions
- POST /api/shipments/{id}/exceptions
- PATCH /api/exceptions/{id}
- Endpoint lain boleh ditambah sesuai desain.

API contract & quality rules

Daftar endpoint saja belum cukup: API harus konsisten, aman, dan dapat diuji.

Request/response

- JSON request/response.
- 200/201 untuk sukses; 401/403 untuk auth; 404 untuk resource; 422 untuk validation.
- Pagination dan filter pada server.
- Error response tidak membocorkan stack trace.
- Postman collection disertakan.

Authorization rules

- Customer hanya dapat melihat shipment miliknya.
- Operator hanya mengubah shipment sesuai izin.
- Admin dapat mengelola konfigurasi.
- Jangan hanya menyembunyikan tombol pada frontend.
- Uji akses tanpa izin melalui automated test.

Tech stack yang disarankan

Gunakan satu stack utama; framework frontend tambahan tidak diwajibkan.

Application

- PHP 8.3+ (sesuaikan dengan versi Laravel yang dipakai).
- Laravel 12+ atau versi stabil yang tersedia di lingkungan kampus.
- Blade templates.
- Tailwind CSS.
- Vanilla JavaScript + Fetch API.

Data & integration

- MySQL 8+.
- Eloquent ORM + migrations/seeders.
- Laravel REST API.
- Laravel Sanctum.
- Laravel HTTP Client.
- Leaflet.js + OpenStreetMap.
- Chart.js.

Engineering & delivery

- Git + GitHub.
- PHPUnit atau Pest.
- Postman.
- Docker (advanced/mandatory bila sudah dilatih).
- GitHub Actions CI/CD (advanced).
- Cloud/VPS deployment.

Architecture guideline

Pisahkan HTTP handling dari business logic; jangan menaruh seluruh logika di controller.



Business services

- ShipmentService • TrackingService
- EtaService • ExceptionService
- Validasi aturan bisnis dan transaksi database.

Persistence & external services

- MySQL melalui Eloquent.
- Weather/geocoding melalui HTTP Client.
- Tangani timeout, error, dan konfigurasi environment.

Security & testing requirements

Security bukan fitur tambahan; ini bagian dari acceptance criteria.

Security wajib

- Password hashing dan auth middleware.
- Authorization di backend.
- Form/request validation.
- CSRF/XSS/SQL injection prevention.
- Mass-assignment protection.
- .env dan API keys tidak masuk GitHub.

Testing wajib

- Authentication: valid/invalid login dan akses tanpa login.
- Shipment CRUD + validation + authorization.
- Tracking event dan invalid status transition.
- ETA/delay rule.
- Minimal beberapa automated API/feature tests.
- Sediakan cara menjalankan test di README.

Milestone pengerjaan mandiri

P8–P15 merupakan milestone kerja mandiri; tidak ada materi kuliah/konsultasi setelah UTS.

P8	Requirement, use case, ERD, architecture, repo, setup/auth
P9	Shipment, carrier, location, role/permission
P10	Tracking events, timeline, status transition
P11	ETA/delay, exception management, REST API
P12	Map + external API integration
P13	Dashboard, charts, search/filter, notification
P14	Security hardening + automated tests
P15	Deployment, English README, API docs, final QA

Milestone adalah panduan manajemen proyek, bukan sesi pengajaran.

Deliverables UAS

Serahkan artefak yang dapat diverifikasi, bukan hanya file source code.

Product & repository

- Public GitHub repository.
- Live deployed application + demo account.
- English README: problem, features, setup, tests, deployment.
- .env.example dan seed/demo data.
- Screenshots dan demo video singkat (opsional).

Technical documentation

- ERD dan data dictionary.
- Architecture diagram.
- Postman collection/API documentation.
- Automated test suite dan test evidence.
- Known limitations dan future work.
- Attribution untuk library, template, dan kode eksternal.

UAS presentation & code ownership interview

Demo membuktikan aplikasi berjalan; interview membuktikan mahasiswa memahami implementasinya.

Live demo

- Jalankan skenario shipment.
- Tambahkan tracking event.
- Tunjukkan timeline/map.
- Picu delay/exception.
- Tunjukkan API dan dashboard.

Code interview

- Dosen memilih file/fungsi secara acak.
- Mahasiswa menjelaskan alur dan keputusan desain.
- Jelaskan authorization dan validation.
- Modifikasi kecil secara langsung.

Contoh pertanyaan

- Mengapa relasi ini hasMany?
- Apa akibat validasi dihapus?
- Bagaimana mencegah status invalid?
- Ubah delay threshold sesuai skenario kelompok.

Anti-copy-paste policy

Menggunakan library/framework diperbolehkan; menyerahkan kode yang tidak dipahami tidak diperbolehkan.

Aturan source code

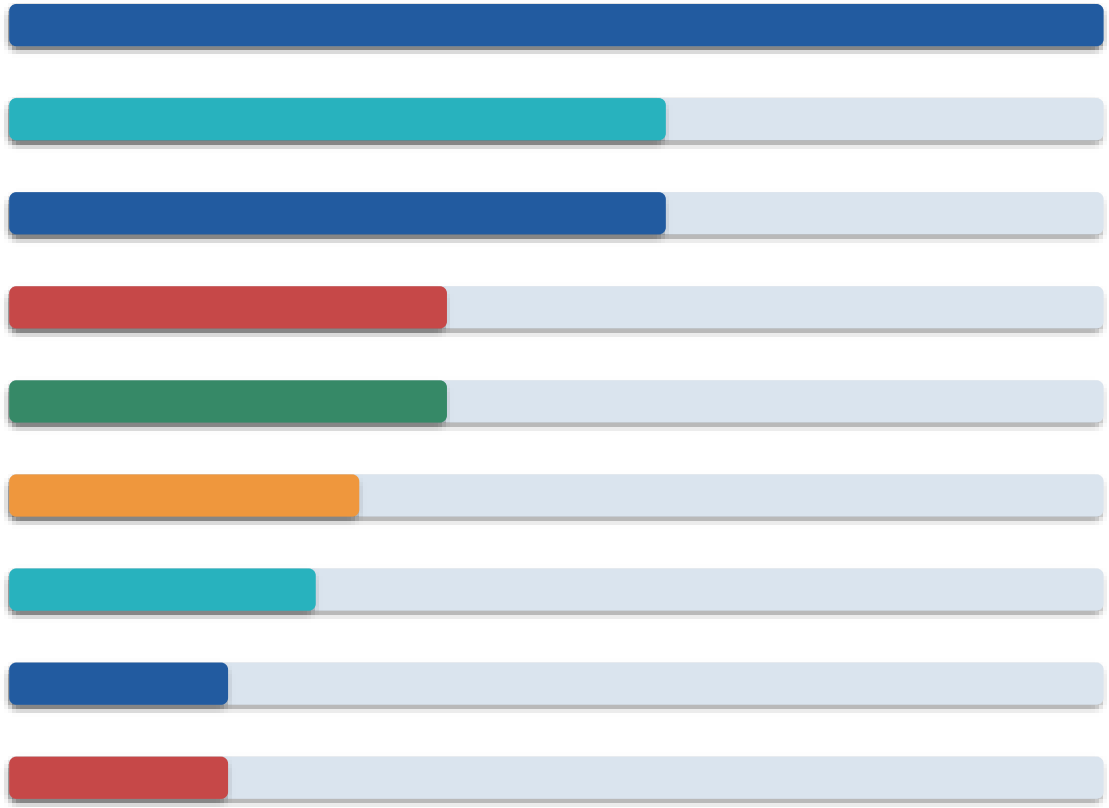
- Semua kelompok wajib mencatat sumber kode/template eksternal.
- Dilarang mengklaim repository orang lain sebagai karya sendiri.
- Commit history harus menunjukkan proses pengembangan.
- Setiap anggota wajib memahami modul yang ditugaskan.
- Kode hasil AI boleh digunakan jika kebijakan dosen mengizinkan, tetapi wajib diverifikasi dan dapat dijelaskan.

Unique group scenario

- Setiap kelompok mendapat parameter/rule berbeda.
- Contoh: delay threshold 12/18/24 jam.
- Carrier, route, exception cases, dan data seed berbeda.
- Dosen dapat memberi perubahan requirement saat interview.
- Penilaian individu dapat berbeda dari nilai kelompok.

Rubrik penilaian UAS

Bobot dapat disesuaikan dengan RPS; rubrik ini menekankan kemampuan engineering.



Total: 100% | Nilai individu dapat disesuaikan berdasarkan interview dan kontribusi Git.

Acceptance criteria: minimum untuk lulus

Gunakan checklist ini sebelum UAS.

Application

- Semua role dan akses berfungsi.
- Shipment lifecycle dan tracking berjalan.
- ETA/delay rule menghasilkan hasil yang benar.
- Map dan external API menangani error.
- Search/filter/pagination bekerja.
- Dashboard memakai data aktual.

Engineering

- Migration dan seeder dapat dijalankan.
- Automated tests dapat dijalankan.
- API memiliki validasi dan authorization.
- Tidak ada secret di repository.
- Live deployment tersedia.
- Dokumentasi cukup untuk setup oleh developer lain.

Contoh scenario unik per kelompok

Dosen membagikan konfigurasi kelompok secara terpisah agar hasil tidak identik.

Group A

- Route: Southeast Asia → Europe
- Delay threshold: 12 jam
- Exception utama: customs hold
- KPI: on-time delivery rate

Group B

- Route: Asia → North America
- Delay threshold: 18 jam
- Exception utama: weather delay
- KPI: average delay duration

Group C

- Route: Southeast Asia → Australia
- Delay threshold: 24 jam
- Exception utama: documentation issue
- KPI: exception resolution time

Angka dan route di atas contoh; dosen dapat membuat konfigurasi berbeda untuk setiap kelompok.

Definition of done

CargoTrace dianggap selesai jika dapat dijalankan, diuji, dijelaskan, dan diakses secara online.

Build

- Fitur mandatory berfungsi.
- Database konsisten.
- Business rules tervalidasi.
- UI responsif.

Verify

- Automated tests lulus.
- Role/access diuji.
- API dapat dicoba.
- Tidak ada secret ter-commit.

Present

- Live URL tersedia.
- GitHub dan English README lengkap.
- Dokumentasi tersedia.
- Anggota mampu menjelaskan kode.

Build it. Test it. Explain it.

Tujuan akhir bukan hanya aplikasi yang berjalan, tetapi bukti kemampuan software engineering yang dapat diperiksa recruiter.

CargoTrace • Laravel • MySQL • REST API • Integration • Testing • Deployment

END OF PROJECT BRIEF